

SD Client API

The sdclilib client library, its functions across four language bindings, and the server status codes.

The SDClient API lets an external application connect to SD, open files, read and write records, execute commands, call subroutines, and manage select lists — all through a single shared library.

The API is a normal way for any account to use SD, not a facility reserved for developers and administrators. A person running a custom GUI program that talks to SD needs API access and may need nothing else.

The library

Four DLLs are installed. They are built from one source: each pair is the same code compiled twice under two output names, so the two files are not identical on disk but behave identically.

64-bit	<code>sdclilib.dll</code> and <code>sdclient.dll</code>
--------	---

32-bit	<code>qmclilib.dll</code> and <code>qmclient.dll</code>
--------	---

The `*clilib` names are what existing applications ask for; the `*client` names are for new work. **Renaming one to the other does not work**: an import library records the DLL name its symbols come from, so an application built against `sdclilib` loads `sdclilib.dll` whatever you call the file on disk. That is why both names are built rather than one being a copy of the other.

The architecture must match the application, not the machine. A 32-bit application on 64-bit Windows needs the 32-bit DLL. The 32-bit build is a shipping deliverable rather than a testing convenience.

WHERE THEY ARE, AND HOW TO USE THEM

C:\Program Files\SD\usr\clients\client64\	<code>sdclilib.dll</code>	<code>sdclient.dll</code>
C:\Program Files\SD\usr\clients\client32\	<code>qmclilib.dll</code>	<code>qmclient.dll</code>

Copy the DLL your application needs either **beside the application's own executable** or into `C:\Windows\System32`. Those are the two supported routes and either works.

`C:\Program Files\SD\usr\bin` is normally on the system PATH, because the installer offers to put it there — the "Add SD Core to the system PATH" task, which is ticked unless you untick it, and it is what makes `sd` run from any directory. **All four client DLLs live in that directory too**, so an application may find one without your having copied anything. That is convenient, and it is not the same as choosing which copy it loads: PATH order decides, and a stale copy earlier on the PATH wins. **Put the DLL where your application will find it deliberately.**

`usr\clients` holds the DLLs **and one import library for each** — `.dll.a` files, GNU-style, for linking rather than loading:

```
client64\    libsdclilib.dll.a  libsdclient.dll.a
client32\    libqmcclilib.dll.a libqmcclient.dll.a
```

They sit beside the DLLs so that everything a client *build* needs is in one place. **No header travels with them:**

`sdclilib.h` reaches every installation at `C:\ProgramData\SD\sdsys\syscom\sdclilib.h`.

All four DLLs also appear in `C:\Program Files\SD\usr\bin`, **beside** `sd.exe`, and each copy is byte-identical to the one under `usr\clients`. The 32-bit pair is there because `usr\bin` is on the PATH, and that is where a **32-bit administrative utility** finds its client. Those copies are the server's own and are not the ones you should be taking — with one exception, which is the next section. **The import libraries are deliberately not in** `usr\bin`: a linker input has no business in a directory that goes on the PATH.

Connection

`SDConnect(host, port, user, pass, account)` over the network, to port **4243**

`SDConnectLocal(account)` on the same machine. Sends no password and never did

`SDConnectUDS` (Unix Domain Socket) appears in the header but is not applicable on Windows. The Windows port supports local and TCP connections only.

SDCONNECTLOCAL HAS A REQUIREMENT THE OTHER DOES NOT

`SDConnectLocal` starts a session by running `sd.exe`, and **it looks for** `sd.exe` **beside itself** — in the directory the loaded DLL came from, not on the PATH.

So a copy of the DLL sitting next to your own application will not make a local connection: there is no `sd.exe` there. Two ways round it:

Load the copy in `usr\bin`it is beside `sd.exe`, which is why that copy exists

Use `SDConnect` insteadconnect to `127.0.0.1` on port 4243 like any other client

`SDConnectLocal` sends no password at all. It takes the identity of the process that called it and checks that account's grants, so the account has to be one the calling Windows user may enter.

The login is SCRAM-SHA-256. A client that sends a password in clear is refused. The server sets a puzzle only someone who knows the password can answer, and the password itself is never sent in any form. The server also proves itself to the client — another program that grabbed the port before SD started cannot pretend to be SD.

Run `modify.password` again for every account that uses the API after upgrading. The stored credentials changed shape and the old ones cannot be converted — the password was never kept anywhere, by design.

A session is confined to its own account. An API session can open everything inside its own account and the shipped SDSYS files every account needs, but cannot open, rename, delete or list anything else.

Server status codes

CONSTANT	VALUE	MEANING
<code>SV_OK</code>	0	success
<code>SV_ON_ERROR</code>	1	an <code>ON_ERROR</code> clause fired
<code>SV_ELSE</code>	2	an <code>ELSE</code> clause fired
<code>SV_ERROR</code>	3	an error occurred
<code>SV_LOCKED</code>	4	the record is locked by another session
<code>SV_PROMPT</code>	5	the command issued a prompt and is waiting for input

Function reference

CONNECTION MANAGEMENT

FUNCTION	RETURNS	DESCRIPTION
<code>SDConnect(host, port, user, pass, account)</code>	Boolean	connect over TCP
<code>SDConnectLocal(account)</code>	Boolean	connect on the same machine
<code>SDConnected()</code>	Boolean	is a session active
<code>SDDisconnect()</code>	none	end the current session
<code>SDDisconnectAll()</code>	none	end all sessions
<code>SDGetSession()</code>	Integer	get the current session number
<code>SDSetSession(session)</code>	Boolean	switch to a session
<code>SDLogto(account)</code>	Boolean	switch to another account

FILE OPERATIONS

FUNCTION	RETURNS	DESCRIPTION
<code>SDOpen(filename)</code>	Integer (file number)	open a file
<code>SDClose(fileNo)</code>	none	close a file
<code>SDRead(fileNo, id, err)</code>	String	read a record
<code>SDReadl(fileNo, id, wait, err)</code>	String	read with a shared lock
<code>SDReadu(fileNo, id, wait, err)</code>	String	read with an update lock
<code>SDWrite(fileNo, id, data)</code>	none	write a record
<code>SDWriteu(fileNo, id, data)</code>	none	write with an update lock
<code>SDDelete(fileNo, id)</code>	none	delete a record
<code>SDDeleteu(fileNo, id)</code>	none	delete with an update lock
<code>SDRecordlock(fileNo, id, updateLock, wait)</code>	none	set a record lock
<code>SDRelease(fileNo, id)</code>	none	release a record lock

FUNCTION	RETURNS	DESCRIPTION
<code>SDMarkMapping(fileNo, state)</code>	none	enable/disable mark mapping

RECORD MANIPULATION

FUNCTION	RETURNS	DESCRIPTION
<code>SDExtract(src, fno, vno, svno)</code>	String	extract a field, value or subvalue
<code>SDIns(src, fno, vno, svno, newdata)</code>	String	insert into a dynamic array
<code>SDDel(src, fno, vno, svno)</code>	String	delete from a dynamic array
<code>SDReplace(src, fno, vno, svno, newdata)</code>	String	replace a field, value or subvalue
<code>SDLocate(item, src, fno, vno, svno, pos, order)</code>	Boolean	find a position for ordered insert

STRING FUNCTIONS

FUNCTION	RETURNS	DESCRIPTION
<code>SDField(str, delim, first, occurrences)</code>	String	extract a field from a delimited string
<code>SDDcount(src, delim)</code>	Long	count delimiters
<code>SDChange(str, old, new, occurrences, start)</code>	String	replace substrings
<code>SDMatch(src, template)</code>	Boolean	match a string against a pattern
<code>SDMatchfield(src, template, component)</code>	String	extract a matched component
<code>SDSubstr(...)</code>	String	substring (language-specific)

COMMAND EXECUTION

FUNCTION	RETURNS	DESCRIPTION
<code>SDExecute(cmnd, err)</code>	String	execute a TCL command
<code>SDRespond(response, err)</code>	String	respond to a prompt from <code>SDExecute</code>
<code>SDEndCommand()</code>	none	end a multi-line command
<code>SDCall(name, argcount, ...)</code>	none	call a catalogued subroutine (pass by reference)
<code>SDCallx(name, argcount, ...)</code>	Integer	call a catalogued subroutine (pass by value)

`SDCall` and `SDCallx` are variadic: 0 to 20 arguments. Each binding provides per-arity wrappers — `SDCall0` through `SDCall20`, and `SDCallx0` through `SDCallx20`.

SELECT LISTS

FUNCTION	RETURNS	DESCRIPTION
<code>SDSelect(fileNo, listNo)</code>	none	select all records in a file
<code>SDSelectIndex(fileNo, indexName, indexValue, listNo)</code>	none	select by index value
<code>SDSelectLeft(fileNo, indexName, listNo)</code>	String	select left of an index cursor
<code>SDSelectRight(fileNo, indexName, listNo)</code>	String	select right of an index cursor
<code>SDSetLeft(fileNo, indexName)</code>	none	position cursor left
<code>SDSetRight(fileNo, indexName)</code>	none	position cursor right
<code>SDReadNext(listNo, err)</code>	String	read next select list id
<code>SDReadList(listNo, err)</code>	String	read the whole list as a dynamic array
<code>SDClearSelect(listNo)</code>	none	clear a select list
<code>SDRelease(fileNo, id)</code>	none	release a read lock

SESSION AND PACKAGE MANAGEMENT

FUNCTION	RETURNS	DESCRIPTION
<code>SDEnterPackage(name)</code>	Boolean	enter a package context
<code>SDExitPackage(name)</code>	Boolean	exit a package context
<code>SDGetArg(argNo)</code>	String	get a command argument

ERROR HANDLING

FUNCTION	RETURNS	DESCRIPTION
<code>SDError()</code>	String	get the last error message
<code>SDDebug(mode)</code>	none	enable/disable debug mode

FUNCTION	RETURNS	DESCRIPTION
<code>SDStatus()</code>	Long	get the last server status code
<code>SDFree(ptr)</code>	none	free a returned pointer

Language bindings

The API headers define bindings for four languages. The function signatures are identical in meaning; the syntax differs.

GAMBAS3

```
Library "./sdclilib"
Extern SDConnect(Host As String, Port As Integer, UserName As String,
    Password As String, Account As String) As Boolean
Extern SDRead(FileNo As Integer, Id As String,
    ByRef Errno As Integer) As String
```

PUREBASIC

```
PrototypeC.l P_SDConnect(host.p-utf8, port.l, user.p-utf8,
    pass.p-utf8, account.p-utf8)
PrototypeC.i P_SDRead(fno.l, id.p-utf8, *err)
```

Per-arity prototypes are provided for `SDCall` and `SDCallx`: `P_SDCall0` through `P_SDCall120`, `P_SDCallx0` through `P_SDCallx20`.

FREE PASCAL / LAZARUS

```
type
    TSDConnectFn = function(Host: PAnsiChar; Port: LongInt;
        User, Pass, Account: PAnsiChar): LongInt; cdecl;
    TSDReadFn = function(FileNo: LongInt; Id: PAnsiChar;
        Err: PLongInt): PAnsiChar; cdecl;
```

A `LoadSdCliLib` function loads the library and resolves all function pointers; `UnloadSdCliLib` releases it.

PYTHON (CTYPES)

```
import ctypes
_lib = ctypes.CDLL('./sdclilib.dll')

SDConnect = _lib.SDConnect
SDConnect.argtypes = [ctypes.c_char_p, ctypes.c_int,
    ctypes.c_char_p, ctypes.c_char_p, ctypes.c_char_p]
SDConnect.restype = ctypes.c_int

SDRead = _lib.SDRead
SDRead.argtypes = [ctypes.c_int, ctypes.c_char_p,
    ctypes.POINTER(ctypes.c_int)]
SDRead.restype = ctypes.c_char_p
```

Per-arity `CFUNCTYPE` definitions are provided for `SDCall` and `SDCallx` wrappers.

What the API cannot do

<code>sh</code> and <code>OS.EXECUTE</code>	refused over the API
Open files outside the account	refused (status 3035 — <i>not permitted</i>)
Reach the credential file	never, and cannot be added
Enumerate accounts	refused; all three failure cases give the same message